

Report Tecnico

Machine Learning Applicato — Problema A: Classificazione

Previsione della sottoscrizione di un deposito a termine

Dataset: UCI Bank Marketing (bank-full)

Piattaforma: Microsoft Fabric · Power BI · Power Query

Progetto: Progetto-Valtellina

Data: 26 giugno 2026

Indice

1	Introduzione e Motivazione del Dataset	2
1.1	Contesto di business	2
1.2	Variabili e variabile target	2
1.3	Ipotesi iniziali e sbilanciamento delle classi	2
1.4	Problemi di qualità del dato previsti	2
2	Analisi Esplorativa dei Dati (EDA)	2
2.1	Statistiche descrittive	3
2.2	Visualizzazioni	3
2.3	Analisi dei valori mancanti	4
3	Pipeline di Data Cleaning e Feature Engineering (Power Query)	5
3.1	Step di trasformazione	5
3.2	Feature engineering: variabili create	6
3.3	Requisiti tecnici Power Query soddisfatti	6
4	Descrizione e Confronto dei Modelli Sviluppati	7
4.1	Modelli baseline (parametri di default)	7
4.2	Ottimizzazione degli iperparametri	7
4.3	Workflow di valutazione e tracking MLflow	7
5	Analisi delle Metriche e Scelta del Modello Migliore	7
5.1	Perché ROC-AUC come metrica guida	8
5.2	Modello scelto: RandomForest_Tuned	8
6	Interpretazione dei Risultati e Conclusioni	10

1. Introduzione e Motivazione del Dataset

Il progetto applica tecniche di Machine Learning supervisionato a un problema di classificazione binaria su dati di marketing bancario, nell'ambito dell'esercitazione "Machine Learning Applicato" su Microsoft Fabric.

1.1 Contesto di business

Il dataset "Bank Marketing" (UCI Machine Learning Repository) raccoglie i risultati di campagne di telemarketing condotte da un istituto bancario portoghese per promuovere la sottoscrizione di depositi a termine (term deposit). Ogni riga rappresenta un contatto telefonico con un cliente, con informazioni demografiche, finanziarie e relative allo storico delle campagne precedenti. L'obiettivo di business è identificare in anticipo i clienti con maggiore probabilità di sottoscrivere il prodotto, per ottimizzare le risorse del call center e aumentare il tasso di conversione delle campagne.

1.2 Variabili e variabile target

Il dataset contiene complessivamente 45.211 record e 17 variabili (16 predittive più il target), di tipo misto numerico e categoriale. La variabile target y è categoriale binaria (sì/no sottoscrizione). Le feature includono variabili demografiche (age, job, marital, education), finanziarie (balance, housing, loan, default) e relative alla campagna corrente e a quelle precedenti (contact, month, campaign, pdays, previous, poutcome). La variabile duration (durata della chiamata), pur disponibile nel dataset originale, è stata esclusa dal training: il suo valore è noto solo a chiamata terminata e introduce data leakage, rendendo il modello inutilizzabile in un contesto predittivo reale (decisione da prendere prima della chiamata).

1.3 Ipotesi iniziali e sbilanciamento delle classi

Ipotesi di partenza: i clienti contattati in mesi specifici, con storico di campagne precedenti positivo (poutcome) e con minore esposizione debitoria (no housing/personal loan) hanno maggiore probabilità di sottoscrizione. Il target è fortemente sbilanciato (circa 88% "no" contro 12% "sì"): tutti i modelli sono stati addestrati con gestione esplicita dello sbilanciamento (class_weight="balanced" su scikit-learn, scale_pos_weight su XGBoost, class weight dedicato su Keras), e la metrica principale di confronto è stata fissata su ROC-AUC invece che sulla sola accuracy, poco informativa su classi sbilanciate.

1.4 Problemi di qualità del dato previsti

- Valori mancanti codificati come stringa "unknown" in più colonne categoriali (es. job, education, contact, poutcome);
- Outlier nelle variabili quasi continue (balance, duration, campaign, pdays);
- Variabili categoriali da codificare numericamente per l'uso nei modelli;
- Necessità di standardizzare le variabili quasi continue per i modelli sensibili alla scala (SVM, KNN, Logistic Regression, rete neurale).

2. Analisi Esplorativa dei Dati (EDA)

L'EDA è stata condotta in due passaggi nel notebook Fabric dedicato (EDA_CLASSIFICAZIONE.ipynb): una prima analisi sul dataset grezzo importato dal Lakehouse, e una seconda analisi di verifica sul dataset pulito, dopo la pipeline di Power Query, per confermare l'effetto delle trasformazioni applicate.

2.1 Statistiche descrittive

Per ogni variabile numerica sono state calcolate media, mediana, deviazione standard, minimo, massimo e quartili. Per le variabili categoriali sono state prodotte tabelle di frequenza. È stata inoltre calcolata la percentuale di valori “unknown” per colonna.

2.2 Visualizzazioni

Per l’analisi delle distribuzioni sono stati prodotti gli istogrammi (con stima di densità KDE) di tutte le variabili, mostrati in Figura 1. Si precisa che, quando in questo report si parla di variabili *quasi continue*, si intendono variabili tecnicamente discrete ma con un numero molto elevato di modalità distinte (es. age, balance, duration, campaign, pdays, previous), che statisticamente vengono trattate come continue.

Due tipologie di grafico previste in fase di analisi non sono state incluse nel report, per le seguenti ragioni metodologiche:

- **Boxplot** — non utilizzati perché poco informativi dal punto di vista visivo: le variabili quasi continue del dataset presentano una modalità nettamente dominante (es. pdays e previous concentrate sul valore convenzionale dei clienti mai contattati), per cui il boxplot finiva per segnalare come outlier tutto ciò che si discostava da quella singola modalità, senza fornire informazione utile;
- **Scatter plot con retta di regressione** — non inseriti perché le variabili quasi continue presentano tra loro una correlazione quasi nulla (inferiore al 10%), quindi i grafici di dispersione non avrebbero evidenziato alcuna relazione significativa.

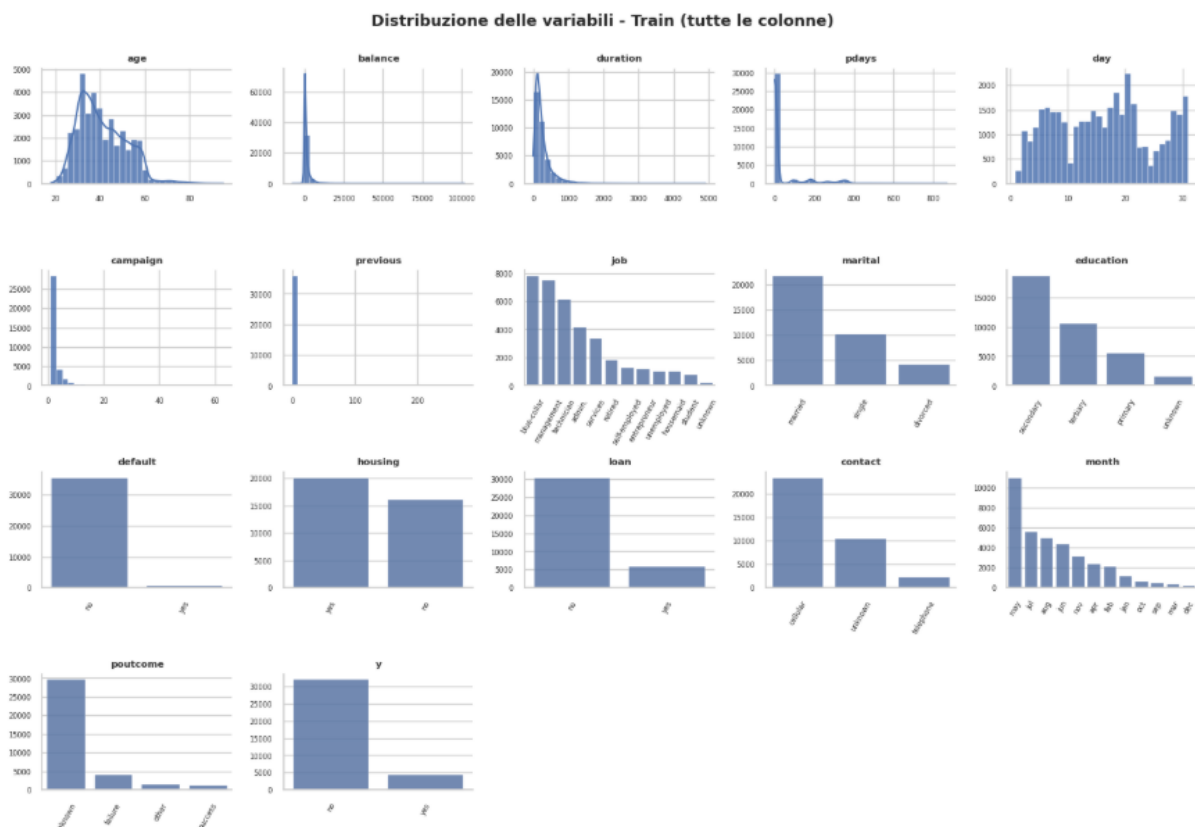


Figura 1 — Distribuzione di tutte le variabili del dataset di train (istogrammi per le numeriche, conteggi di frequenza per le categoriali). Si nota la forte asimmetria di balance, duration, campaign e previous, e lo sbilanciamento del target y (~88% “no”).

Matrice di associazione

Per misurare la relazione tra ciascuna variabile e il target (e tra le variabili stesse) è stata costruita un'unica matrice di associazione che combina due misure a seconda del tipo di coppia di variabili: il coefficiente di correlazione di **Pearson** (in valore assoluto) per le coppie di variabili *quasi continue*, e l'indice di **Cramér's V** (derivato dal test del Chi-quadrato) per le coppie che coinvolgono variabili *categoriali*. In questo modo si misura l'intensità dell'associazione senza assumere una relazione lineare dove non avrebbe senso (variabili categoriali). I valori vanno da 0 (nessuna associazione) a 1 (associazione perfetta).

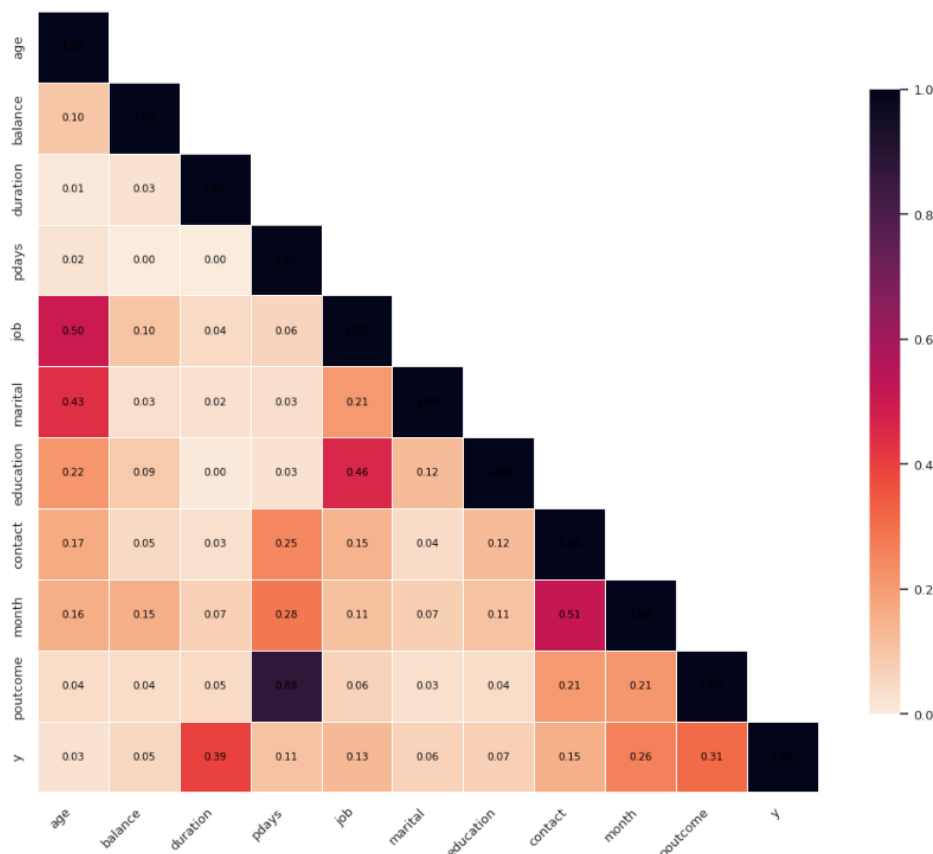


Figura 2 — Matrice di associazione (Pearson per le quasi continue, Cramér's V per le categoriali). Rispetto al target *y*, le associazioni più forti sono *duration* (0,39), *poutcome* (0,31) e *month* (0,26). Si nota la fortissima associazione *pdays*–*poutcome* (0,88), spiegata dal fatto che entrambe codificano lo storico di contatto del cliente.

*Nota: la variabile **duration** compare in questa analisi esplorativa (e risulta la più associata al target) ma è stata poi esclusa dal training dei modelli per evitare data leakage: il suo valore è noto solo a chiamata conclusa e non sarebbe disponibile al momento della decisione (cfr. Sezione 1.2 e Sezione 4).*

2.3 Analisi dei valori mancanti

Il dataset non presenta valori nulli in senso tecnico (NaN), ma codifica il missing come categoria testuale “unknown” in colonne come *job*, *education*, *contact* e *poutcome*. Il pattern è compatibile con MAR (Missing At Random condizionato): ad esempio, *poutcome*=“unknown” è sistematicamente associato ai clienti senza campagne precedenti (*previous*=0), quindi non è un dato mancante casuale ma una conseguenza diretta di un'altra variabile osservata. La strategia di imputazione scelta è stata l'imputazione per moda, calcolata per gruppo (raggruppando per *job*), applicata in Power Query.

La tabella seguente riporta le sole colonne con valori “unknown” presenti; tutte le altre variabili del dataset hanno conteggio 0.

Variabile	Unknown count	Unknown %
poutcome	29.560	81,73
contact	10.385	28,71
education	1.483	4,10
job	233	0,64

Tabella 1 — Conteggio e percentuale di valori “unknown” per variabile (dataset di train). Le altre 13 variabili non presentano valori mancanti.

Osservazione sulla strategia: poutcome ha l’81,73% di valori “unknown”, ma questo non è un vero dato mancante: corrisponde ai clienti mai contattati prima (`pdays = -1`, `previous = 0`). Per questo motivo non è stato imputato ma gestito tramite feature engineering dedicato (cfr. Sezione 3.2), mentre l’imputazione per moda è stata applicata alle colonne con missing rate basso e realmente casuale (education, job).

Nota sull’imputazione del test set: per evitare data leakage, la moda di imputazione andrebbe in teoria calcolata sul solo train e poi applicata al test. In pratica, su Fabric/Power BI risulta complesso imputare il test riferendosi alle statistiche calcolate sul train. Si è quindi scelto di imputare il test con la propria moda, dopo aver verificato che la moda del test coincide con quella del train: in questo modo il risultato è identico a quello che si otterrebbe propagando la moda del train, senza introdurre leakage.

Identificazione degli outlier

Gli outlier sono stati individuati con il metodo dell’IQR (Inter-Quartile Range), considerando outlier i valori esterni all’intervallo $[Q_1 - 1,5 \cdot IQR, Q_3 + 1,5 \cdot IQR]$. L’analisi è stata limitata alle variabili quasi continue.

Colonna	Q1	Q3	IQR	Lower	Upper	Outliers	%
pdays	-1,0	-1,0	0,0	-1,0	-1,0	6.612	18,28
previous	0,0	0,0	0,0	0,0	0,0	6.612	18,28
balance	70,0	1.429,0	1.359,0	-1.968,5	3.467,5	3.788	10,47
duration	103,0	318,0	215,0	-219,5	640,5	2.589	7,16
campaign	1,0	3,0	2,0	-2,0	6,0	2.467	6,82
age	33,0	48,0	15,0	10,5	70,5	390	1,08
day	8,0	21,0	13,0	-11,5	40,5	0	0,00

Tabella 2 — Outlier per variabile quasi continua (metodo IQR, dataset di train). I valori “anomali” di `pdays` e `previous` non sono veri outlier statistici ma riflettono il codice convenzionale -1/0 dei clienti mai contattati prima, gestito tramite feature engineering anziché rimozione.

3. Pipeline di Data Cleaning e Feature Engineering (Power Query)

La preparazione dei dati è stata svolta interamente in Power Query (Dataflow Gen2) all’interno dell’ecosistema Fabric, secondo la pipeline tracciata nel pannello “Passaggi applicati”.

3.1 Step di trasformazione

- Rimozione dei duplicati sul dataset grezzo;

- Imputazione dei valori “unknown” in `education` tramite moda calcolata per gruppo (raggruppando per `job`), per evitare di introdurre un bias legato alla moda globale; la moda del train coincide con quella del test, quindi l’imputazione è coerente su entrambi i set senza introdurre data leakage (cfr. Sezione 2.3);
- Trasformazione log a segno preservato (sign-preserving log transform) sulla variabile `balance`, che presenta valori negativi (scoperti di conto) e una distribuzione fortemente asimmetrica;
- Standardizzazione (z-score) delle variabili numeriche quasi continue, calcolata usando solo le statistiche del train per evitare data leakage verso il test;
- Encoding delle variabili binarie testuali (yes/no) in formato numerico 0/1;
- One-hot encoding k-1 delle variabili categoriali multi-classe, tramite una funzione M riutilizzabile (`OneHotEncodeKMinus1`), per evitare collinearità perfetta tra le dummy generate;
- Creazione di feature derivate sullo storico di contatto (`previous_contacted`, `recall`), descritte in dettaglio nella Sezione 3.2.

3.2 Feature engineering: variabili create

Oltre alle trasformazioni di pulizia, sono state costruite alcune feature derivate per esplicitare informazioni latenti nel dataset originale. In particolare, sono stati creati due flag binari sullo storico di contatto del cliente, che nel dataset grezzo è codificato in modo poco trasparente (tramite valori convenzionali come `pdays = -1` o `previous = 0`):

- `previous_contacted` — flag binario derivato da `previous` (numero di contatti effettuati prima della campagna attuale). Vale **1** se `previous` $\neq$ 0 (almeno un contatto pregresso), **0** se `previous` = 0 (nessun contatto precedente);
- `recall` — flag binario derivato da `pdays` (giorni trascorsi dall’ultimo contatto di una campagna precedente, dove -1 è il codice convenzionale per “mai contattato”). Vale **1** se `pdays` $\neq$ -1 (cliente con un valore reale di `pdays`, quindi già contattato in passato), **0** se `pdays` = -1 (mai contattato prima).

Entrambe le feature trasformano un’informazione codificata in modo ambiguo (valori sentinella -1/0) in flag espliciti e direttamente utilizzabili dai modelli. La loro utilità è confermata a posteriori dalla feature importance del modello migliore (cfr. Sezione 5.2), dove `previous_contacted` e `recall` figurano tra le variabili più rilevanti. Le variabili categoriali multi-classe (`job`, `marital`, `education`, `contact`, `month`, `poutcome`) sono state inoltre espanse in dummy 0/1 tramite one-hot encoding k-1, generando colonne come `month_may`, `job_blue-collar`, `contact_unknown`, ecc.

3.3 Requisiti tecnici Power Query soddisfatti

- Più di 3 step di trasformazione tracciati e documentati nel pannello dei passaggi applicati;
- Colonna personalizzata con formula M dedicata (funzione di one-hot encoding k-1 e trasformazione log a segno preservato);
- Split train/test (80/20, seed fisso 42) eseguito prima di calcolare statistiche di media, moda e deviazione standard, per evitare leakage tra le due tabelle `bank_full_train` e `bank_full_test`;
- Dataset pulito esportato come tabelle Delta nel Lakehouse (`bank_full_train_cleaned`, `bank_full_test_clean`), pronte per il consumo dai notebook di modellazione.

4. Descrizione e Confronto dei Modelli Sviluppati

Il notebook MODEL_CLASSIFICAZIONE.ipynb implementa un workflow standardizzato: caricamento del dataset pulito dal Lakehouse, esclusione di duration per data leakage, training con parametri di default, valutazione sul test set mai visto in training, tuning degli iperparametri sui due modelli più promettenti, e tracciamento di ogni run in MLflow (parametri, metriche, modello serializzato).

4.1 Modelli baseline (parametri di default)

- Logistic Regression — baseline lineare, `class_weight="balanced"`;
- Decision Tree Classifier — `class_weight="balanced"`;
- Random Forest Classifier — `class_weight="balanced"`;
- K-Nearest Neighbors — non supporta `class_weight`, monitorato sulle metriche;
- Support Vector Machine (kernel RBF, `probability=True`) — addestrata su un campione stratificato di 8.000 righe del train (stessa proporzione di classi) per limitare il costo computazionale $O(n^2)$ – $O(n^3)$, valutata comunque sull'intero test set;
- XGBoost Classifier — gestione dello sbilanciamento tramite `scale_pos_weight`;
- Rete neurale densa (Keras) — architettura 128-64-32-1, attivazioni ReLU, output sigmoid, Dropout 0,2 tra i layer per limitare l'overfitting, EarlyStopping su `val_loss` (`patience=3`, `restore_best_weights`) per interrompere il training quando la loss di validazione smette di migliorare.

4.2 Ottimizzazione degli iperparametri

RandomForest e XGBoost (i due modelli ad albero più promettenti) sono stati ottimizzati con RandomizedSearchCV, 20 combinazioni (`n_iter=20`), 5-fold cross-validation, `scoring="roc_auc"`, sullo spazio di ricerca di `n_estimators`, `max_depth`, `min_samples_split/leaf` e `max_features` (RandomForest) o `learning_rate`, `max_depth`, `subsample` e `colsample_bytree` (XGBoost).

4.3 Workflow di valutazione e tracking MLflow

Ogni run registra in MLflow: tutti gli iperparametri del modello, le metriche su Accuracy, Precision, Recall, F1 e ROC-AUC sul test set, il modello serializzato e — per il modello migliore — i grafici diagnostici (matrice di confusione, curva ROC, feature importance) come artefatti. I due modelli finali (il migliore tra scikit-learn/XGBoost e il modello Keras) sono stati registrati nel Fabric ML Model Registry.

5. Analisi delle Metriche e Scelta del Modello Migliore

La tabella seguente riassume le metriche di tutti i modelli sul test set, ordinate per ROC-AUC decrescente (dati dalla tabella `metriche_classificazione`, salvata nel Lakehouse e usata dalla dashboard Power BI).

Modello	Accuracy	Precision	Recall	F1-Score	ROC-AUC
RandomForest_Tuned	0,80	0,32	0,59	0,42	0,77
DeepLearning_Keras	0,79	0,31	0,58	0,40	0,76
XGBoost_Tuned	0,74	0,26	0,65	0,37	0,76
SVM	0,76	0,28	0,60	0,38	0,75
XGBoost	0,79	0,30	0,57	0,39	0,75
RandomForest	0,88	0,55	0,16	0,25	0,75
LogisticRegression	0,72	0,25	0,65	0,36	0,75
KNN	0,88	0,47	0,18	0,26	0,68
DecisionTree	0,82	0,25	0,25	0,25	0,57

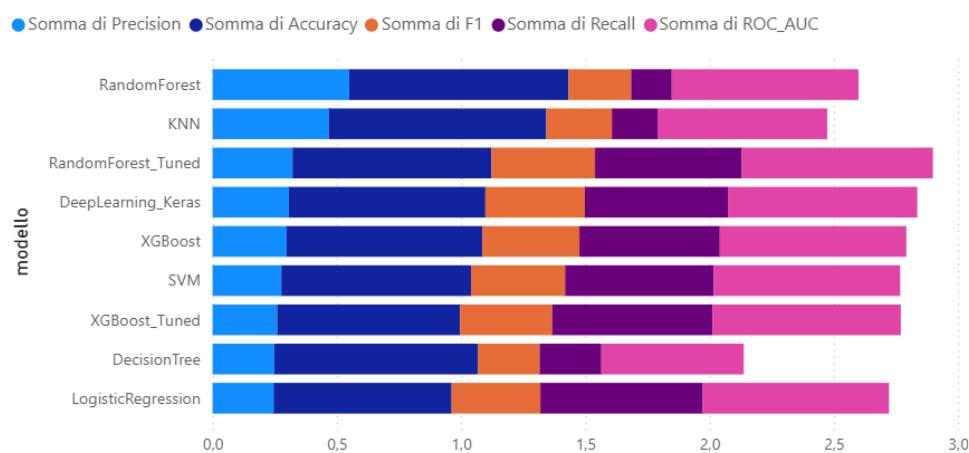


Figura 3 — Confronto grafico delle metriche per modello (dashboard Power BI)

5.1 Perché ROC-AUC come metrica guida

Con un target sbilanciato ($\sim 88\%/12\%$), l'accuracy è ingannevole: un modello che predice sempre "no" otterrebbe già circa 0,88 di accuracy senza alcuna capacità predittiva utile. Si nota infatti che RandomForest e KNN hanno accuracy elevata (0,88) ma recall molto basso (0,16–0,18): identificano pochissimi clienti realmente interessati, il che li rende poco utili per l'obiettivo di business. ROC-AUC misura invece la capacità del modello di discriminare correttamente tra le due classi a qualsiasi soglia di decisione, ed è la metrica scelta come criterio primario di confronto.

5.2 Modello scelto: RandomForest_Tuned

RandomForest_Tuned ottiene il miglior compromesso tra le metriche: ROC-AUC più alta (0,77) insieme a un recall (0,59) nettamente superiore alla versione non ottimizzata (0,16), a fronte di una riduzione contenuta di accuracy (da 0,88 a 0,80). Il tuning degli iperparametri ha quindi spostato il modello verso un comportamento molto più utile dal punto di vista di business, a discapito di una metrica (accuracy) poco rilevante su questo target sbilanciato.

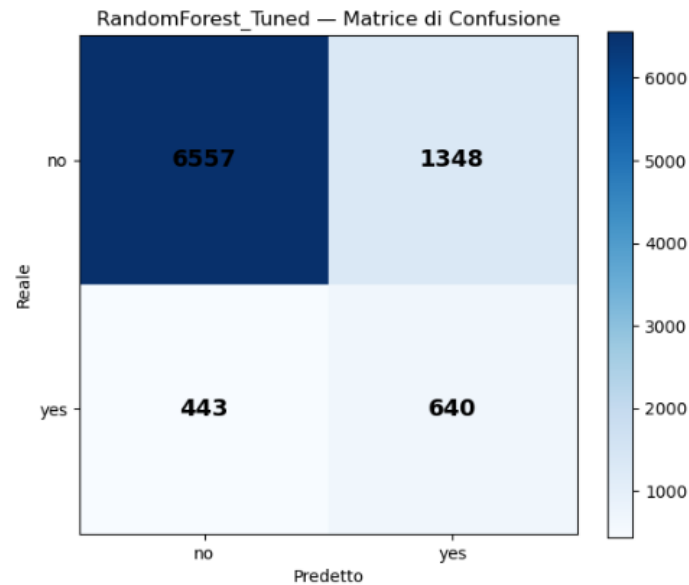


Figura 4 — Matrice di confusione del modello scelto, *RandomForest_Tuned* (test set, $n=8.988$; numeri reali da Lakehouse/Power BI)

Lettura della matrice: il modello identifica correttamente 640 clienti realmente interessati (TP) su 1.083 (recall 0,59), a fronte di 1.348 falsi positivi e 443 falsi negativi. Per una campagna di marketing, un recall più alto a fronte di una precision moderata (0,32) è generalmente preferibile, poiché il costo di un falso positivo (chiamata a un cliente non interessato) è basso rispetto al costo di un falso negativo (cliente interessato non contattato).

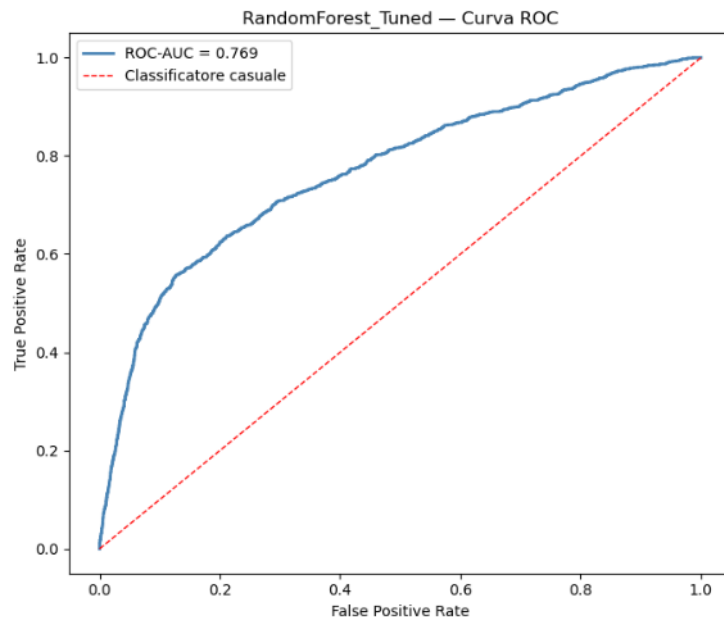


Figura 5 — Curva ROC del modello scelto, *RandomForest_Tuned* (ROC-AUC = 0,769 sul test set). La curva resta nettamente sopra la diagonale del classificatore casuale su tutto l'intervallo, confermando una buona capacità discriminativa.

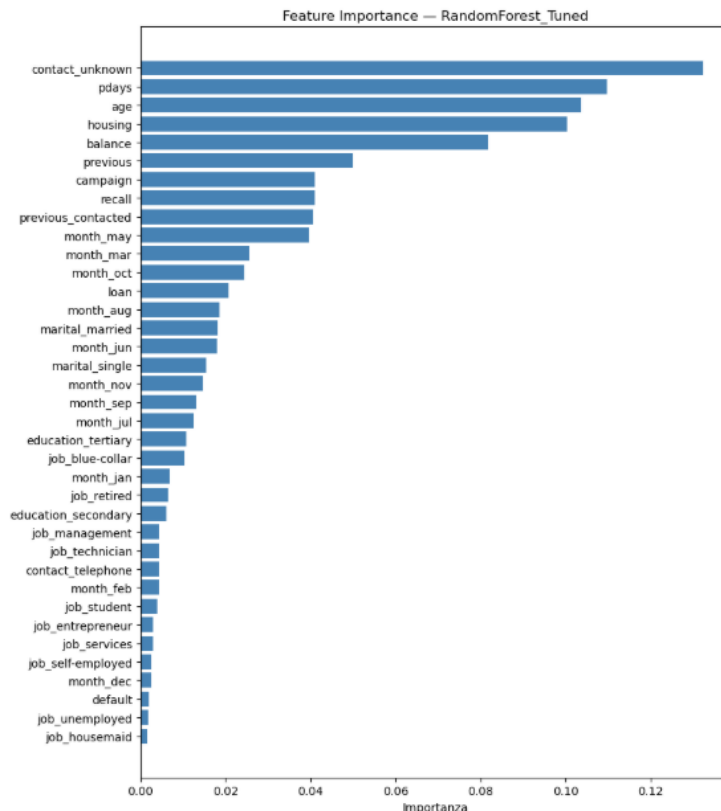


Figura 6 — Feature importance del modello scelto, *RandomForest_Tuned* (*duration* esclusa dal training per data leakage).

Le feature più rilevanti per il modello sono `contact_unknown` (canale di contatto non noto), `pdays` (giorni dall'ultimo contatto di una campagna precedente), `age`, `housing` e `balance`. Compiono ai primi posti anche le due feature engineerizzate `recall` e `previous_contacted` (cfr. Sezione 3.2): la loro presenza tra le variabili più importanti conferma a posteriori l'utilità di aver reso esplicito lo storico di contatto del cliente. Nel complesso il modello privilegia le variabili legate allo storico di contatto e al profilo finanziario rispetto alle sole variabili demografiche di base, coerentemente con l'ipotesi iniziale.

6. Interpretazione dei Risultati e Conclusioni

- La rimozione di `duration` (data leakage) ha ridotto il ROC-AUC del modello migliore da circa 0,91 a circa 0,77: una riduzione attesa e necessaria per ottenere un modello realmente utilizzabile in produzione, dove la durata della chiamata non è nota al momento della decisione;
- La gestione esplicita dello sbilanciamento delle classi (`class_weight / scale_pos_weight`) è stata determinante: senza di essa i modelli tendono a privilegiare l'accuracy ignorando la classe minoritaria, come mostrato dal basso recall di RandomForest e KNN nella configurazione base;
- Il modello Deep Learning (Keras) ottiene performance comparabili ai modelli ad albero ottimizzati (ROC-AUC 0,76), confermando che per questo dataset tabellare i modelli ad albero rimangono competitivi rispetto a una rete neurale, a fronte di un costo di training e interpretabilità inferiore;
- Limiti: il dataset proviene da una singola banca e un singolo contesto geografico/temporale (Portogallo), quindi la generalizzazione ad altri contesti di marketing bancario non è garantita; il recall del modello scelto (0,59) lascia ancora un 41% di clienti potenzialmente interessati non identificati.